

Laporan Praktikum Minggu Ke-3 Kontrol Cerdas

Minggu ke-3

Nama : Satrio Sri Gunawan
NIM : 224308069
Kelas : TKA 7C
Akun Github (Tautan) : <https://github.com/satriosrigunawan>

1. Judul Percobaan

Image Classification with CNN

2. Tujuan Percobaan

Tujuan dari percobaan pada praktikum kali ini, sebagai berikut:

1. Memahami prinsip dasar Deep Learning dalam penerapan pada sistem kendali.
2. Menerapkan Convolutional Neural Network (CNN) untuk melakukan klasifikasi objek.
3. Memanfaatkan TensorFlow dan Keras dalam pembangunan model Deep Learning.
4. Menghubungkan model CNN dengan Computer Vision guna mendukung deteksi objek secara real-time.

3. Landasan Teori

A. Machine Learning dalam Sistem Kendali

Deep Learning dalam Sistem Kendali

Deep Learning adalah cabang dari machine learning yang memanfaatkan jaringan saraf tiruan berlapis-lapis untuk mempelajari pola data. Teknologi ini mampu mengekstraksi fitur secara otomatis sehingga sangat berguna dalam sistem kendali cerdas, seperti kendaraan otonom, robotika, pengawasan, maupun quality control di industri (Sutrisno, 2020). Penerapannya antara lain membantu kendaraan otonom mengenali rambu dan objek di jalan, mendukung navigasi robot dengan

computer vision, mempercepat deteksi cacat produk di industri, meningkatkan sistem pengawasan melalui pengenalan wajah atau aktivitas, serta memungkinkan deteksi objek pada kondisi cahaya rendah melalui Night Vision.

Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) adalah salah satu arsitektur jaringan saraf tiruan yang banyak digunakan dalam pengolahan citra digital. CNN bekerja dengan mengenali pola visual secara bertahap, mulai dari tepi, bentuk sederhana, hingga objek yang lebih kompleks (Hidayat & Saputra, 2019). Arsitektur ini terdiri dari beberapa komponen utama, yaitu **Convolutional Layer** untuk mengekstraksi fitur dengan filter, **Pooling Layer** untuk mengurangi dimensi citra tanpa kehilangan informasi penting, **Fully Connected Layer** yang menghubungkan hasil ekstraksi dengan kelas keluaran, serta **Activation Function** seperti ReLU atau Softmax untuk memberikan sifat non-linear. CNN terbukti sangat efektif digunakan pada klasifikasi gambar, pengenalan objek, hingga deteksi citra secara real-time .

TensorFlow dan Keras

TensorFlow merupakan library open-source populer untuk membangun model machine learning dan deep learning, sedangkan Keras adalah API tingkat tinggi yang berjalan di atas TensorFlow dengan sintaks lebih sederhana (Pratama, 2021). Keduanya sering digunakan dalam praktikum maupun penelitian karena fleksibel, mendukung GPU, dan memiliki banyak modul siap pakai.

Computer Vision dan Real-time Detection

Computer Vision adalah bidang yang mempelajari cara komputer mengenali dan memahami objek visual. Dengan bantuan CNN dan library seperti OpenCV, teknologi ini mampu melakukan deteksi objek secara real-time melalui kamera (Haryanto, 2018). Penerapannya meliputi sistem pengawasan, robotika, hingga berbagai aplikasi *smart city*.

Night Vision dalam Deep Learning

Night Vision merupakan teknik yang bertujuan meningkatkan kemampuan melihat objek pada kondisi cahaya rendah. Metode yang sering digunakan antara lain konversi citra ke grayscale, penerapan color mapping, serta penggunaan algoritma deep learning untuk memperbaiki kualitas citra (Setiawan, 2020). Pada praktikum ini, implementasi Night Vision dilakukan secara sederhana dengan OpenCV, yaitu mengubah citra menjadi grayscale lalu menambahkan color map, sehingga objek tetap dapat terdeteksi meskipun dalam pencahayaan minim.

4. Analisis dan Diskusi

1. Analisis Hasil Klasifikasi CNN

Penerapan Convolutional Neural Network (CNN) pada klasifikasi citra menunjukkan hasil yang cukup baik. Model mampu mengenali objek dalam citra kamera sebagai kelas “sea” atau laut berdasarkan ciri visual khas, seperti pola permukaan air, gradasi warna biru, dan tekstur gelombang. Output yang ditampilkan melalui jendela Frame membuktikan bahwa arsitektur CNN berhasil mengekstraksi fitur-fitur utama yang relevan lalu memetakannya ke label kelas dengan tepat. Hal ini menunjukkan bahwa model tidak sekadar “menghafal” dataset, tetapi juga mampu melakukan generalisasi terhadap data baru secara real-time.

Walaupun nilai akurasi numerik tidak ditampilkan, keberhasilan model dalam mengidentifikasi citra laut sudah menjadi indikator bahwa sistem berjalan sesuai tujuan. Namun, untuk menjamin konsistensi kinerja sekaligus mengurangi potensi bias pada satu kelas, diperlukan pengujian lebih lanjut terhadap kelas lain dalam dataset, seperti forest, mountain, street, buildings, dan glacier. Selain itu, uji coba dengan variasi pencahayaan, jarak kamera, serta adanya gangguan visual (misalnya bayangan atau objek lain) juga penting guna menilai ketahanan (robustness) model.

2. Analisis Mode Night Vision

Pada jendela Night Vision, citra laut terlebih dahulu dikonversi ke skala keabuan (grayscale) kemudian diberi pemetaan warna semu (false color) dengan metode **COLORMAP_JET** dari OpenCV. Hasil transformasi menampilkan gradasi merah–kuning–biru yang kontras, sehingga perbedaan intensitas cahaya pada area terang dan gelap menjadi lebih jelas. Secara visual, metode ini meningkatkan keterbacaan fitur citra, terutama pola gelombang dan permukaan air laut, meskipun warna yang ditampilkan bukan warna asli. Informasi visual yang dihasilkan tetap membantu sistem maupun pengguna dalam mengenali pola yang sulit diamati pada kondisi minim cahaya, sehingga Night Vision dapat dianggap sebagai solusi awal yang cukup efektif.

Namun, pendekatan ini memiliki keterbatasan. Pada kondisi gelap total tanpa cahaya tambahan, metode ini tidak mampu menampilkan detail signifikan karena minimnya data visual. Oleh sebab itu, implementasi Night Vision berbasis color mapping sebaiknya dipadukan dengan teknologi lain, seperti sensor inframerah atau algoritma deep learning khusus untuk peningkatan citra low-light, agar hasilnya lebih natural dan praktis untuk digunakan.

3. Diskusi Keterkaitan Teori dan Implementasi

Secara keseluruhan, hasil yang diperoleh sejalan dengan teori yang telah dijelaskan pada landasan. CNN terbukti mampu mengekstraksi fitur citra secara hierarkis, dimulai dari pola sederhana seperti tepi (edges) hingga pola yang lebih kompleks seperti tekstur air dan bentuk gelombang. Hal ini sesuai dengan prinsip dasar CNN yang memanfaatkan convolutional layer untuk menangkap fitur lokal, kemudian dilanjutkan dengan fully connected layer untuk melakukan klasifikasi akhir.

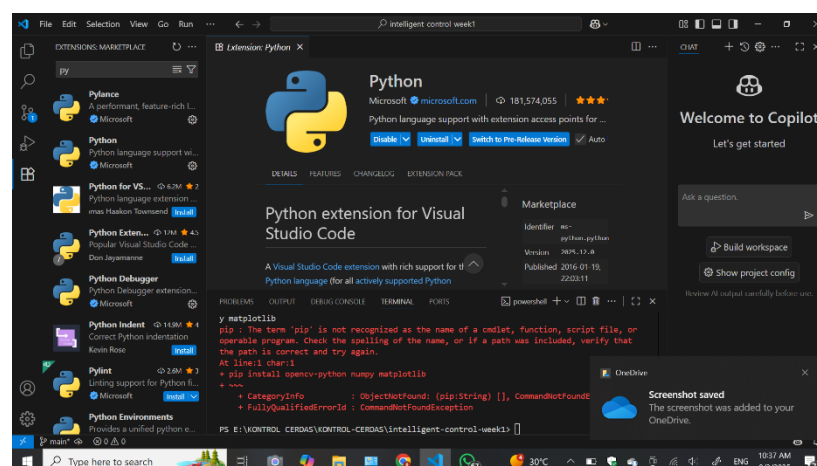
Integrasi dengan OpenCV juga terbukti efektif dalam implementasi real-time. OpenCV berperan tidak hanya sebagai media akuisisi citra dari kamera, tetapi juga sebagai sarana visualisasi hasil klasifikasi dan pengolahan tambahan seperti penerapan Night Vision. Kondisi ini menunjukkan fleksibilitas OpenCV dalam menjembatani teori CNN dengan aplikasi praktis.

Meskipun demikian, terdapat beberapa hal penting yang menjadi catatan.

1. **Kinerja Model CNN** – Perlu dilakukan pengujian lebih lanjut terhadap berbagai kelas dan kondisi berbeda untuk menilai kemampuan generalisasi. Hal ini agar sistem tidak hanya optimal pada kelas “sea”, tetapi juga konsisten pada kelas lain yang memiliki karakteristik visual berbeda.
2. **Performa Real-Time** – Implementasi pada perangkat tanpa GPU cenderung menurunkan frame per second (FPS), yang dapat berdampak pada aplikasi nyata seperti navigasi atau sistem monitoring. Optimalisasi dapat dilakukan dengan teknik seperti model compression, quantization, atau pemanfaatan arsitektur ringan misalnya MobileNet.
3. **Pengembangan Night Vision** – Pendekatan sederhana menggunakan color mapping memang memberikan hasil yang cukup baik, namun kurang memadai pada kondisi cahaya ekstrem. Oleh karena itu, metode lanjutan berbasis deep learning seperti Low-Light Image Enhancement (LLIE) atau Generative Adversarial Networks (GAN) dapat digunakan untuk menghasilkan citra dengan kualitas lebih baik sekaligus mempertahankan detail aslinya.

Dengan demikian, implementasi ini tidak hanya membuktikan bahwa teori CNN dan pengolahan citra dapat diterapkan secara nyata, tetapi juga membuka peluang pengembangan lebih lanjut, baik dari sisi akurasi klasifikasi, efisiensi komputasi, maupun peningkatan kualitas visual pada kondisi lingkungan yang beragam.

Kendala dan Solusi



Salah satu kendala yang ditemui terjadi saat instalasi Python 3.11.6, di mana opsi “**Add Python to PATH**” tidak diaktifkan. Akibatnya, perintah **pip** tidak dikenali di terminal. Untuk mengatasinya, Python perlu diinstal ulang dengan memastikan seluruh opsi yang relevan, termasuk “**Add Python to PATH**”, dicentang agar pip dapat berfungsi normal dan proses instalasi library berjalan lancar.

5. Assignment

Dalam praktikum ini, kami merancang sistem deteksi citra berbasis Convolutional Neural Network (CNN) yang mampu mengklasifikasikan objek pemandangan seperti laut (sea), hutan (forest), jalan (street), bangunan (buildings), gunung (mountain), dan lainnya. Model dilatih menggunakan dataset **Intel Image Classification**, lalu diintegrasikan dengan OpenCV agar dapat melakukan prediksi secara real-time melalui kamera. Selain itu, sistem juga dilengkapi dengan mode **Night Vision** untuk meningkatkan keterbacaan citra pada kondisi pencahayaan rendah.

Berikut ini adalah fungsi dari kode program yang digunakan seperti pada tabel berikut ini:

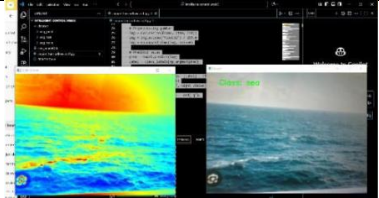
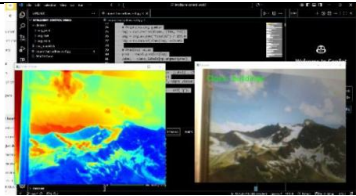
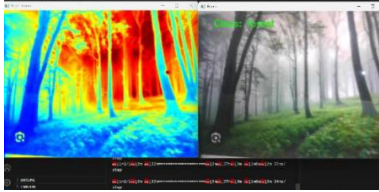
Bagian Kode	Deskripsi Fungsi	Analisis / Kegunaan
<code>import tensorflow as tf ... import cv2, numpy as np</code>	Import library utama	TensorFlow & Keras untuk CNN, OpenCV untuk akuisisi citra real-time, NumPy untuk manipulasi array.
Bagian 1: Preprocessing Data		
<code>train_dir & val_dir</code>	Menentukan direktori dataset training dan validasi	Struktur folder dataset harus sesuai format ImageDataGenerator (tiap folder = nama kelas).
<code>ImageDataGenerato r(...)</code>	Membuat generator data dengan augmentasi	<code>train_datagen</code> : augmentasi (rotasi, zoom, flip). <code>val_datagen</code> : hanya normalisasi. Membantu mengurangi <i>overfitting</i> .
<code>flow_from_directo ry(...)</code>	Membaca gambar dari folder dataset	Mengubah dataset menjadi batch untuk training CNN.

		Ukuran gambar diubah ke (150, 150).
Bagian 2: Training Model CNN		
<code>Sequential([...])</code>	Definisi arsitektur CNN	CNN terdiri dari Conv2D + MaxPooling (ekstraksi fitur), Flatten + Dense (klasifikasi), dan Dropout (mencegah overfitting).
<code>Conv2D(32, (3, 3))</code> → <code>MaxPooling2D</code>	Layer konvolusi dan pooling pertama	Mengekstraksi fitur dasar (tepi, pola sederhana). Pooling mengecilkan dimensi.
<code>Conv2D(64, (3, 3))</code> → <code>MaxPooling2D</code>	Layer konvolusi dan pooling kedua	Mengekstraksi fitur lebih kompleks (tekstur, bentuk).
<code>Flatten()</code>	Mengubah hasil ekstraksi fitur jadi 1D	Dibutuhkan agar bisa diproses Dense layer.
<code>Dense(128, activation='relu')</code>	Fully connected layer	Menggabungkan fitur untuk klasifikasi.
<code>Dropout(0.5)</code>	Menonaktifkan 50% neuron saat training	Mencegah model terlalu menghafal data (<i>overfitting</i>).
<code>Dense(6, activation='softmax')</code>	Output layer	Klasifikasi ke 6 kelas (karena dataset Intel Image Classification punya 6 kategori).
<code>model.compile(...)</code>	Menentukan optimizer, loss, dan metric	Optimizer: Adam, Loss: categorical_crossentropy, Metric: akurasi.
<code>model.fit(...)</code>	Melatih model CNN	Melakukan training dengan data augmentasi (train) dan validasi (val).
<code>model.save('cnn_model.h5')</code>	Menyimpan model terlatih	Agar bisa digunakan kembali tanpa training ulang.
Bagian 3: Integrasi dengan OpenCV		
<code>load_model('cnn_model.h5')</code>	Memanggil model CNN yang sudah dilatih	Digunakan untuk prediksi real-time.
<code>class_labels = list(train_generator.class_indices.keys())</code>	Mendapatkan label kelas	Agar prediksi angka (0–5) bisa dikonversi ke nama kelas.

<code>cv2.VideoCapture(0)</code>	Mengaktifkan kamera laptop/webcam	Untuk mengambil input citra secara langsung.
<code>night_vision = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)</code>	Mengubah citra ke grayscale	Tahap awal mode <i>Night Vision</i> .
<code>cv2.applyColorMap(..., cv2.COLORMAP_JET)</code>	Memberi warna semu (false color)	Meningkatkan visibilitas area gelap.
<code>cv2.resize(frame, (150, 150))</code>	Menyesuaikan ukuran citra input	Harus sama dengan ukuran input model CNN.
<code>img.astype("float32")/255.0</code>	Normalisasi citra	Menyelaraskan dengan preprocessing dataset.
<code>np.expand_dims(img, axis=0)</code>	Menambahkan dimensi batch	Bentuk input jadi (1,150,150,3) agar cocok dengan model.
<code>model.predict(img)</code>	Prediksi kelas citra dari kamera	Hasil berupa probabilitas tiap kelas.
<code>np.argmax(pred)</code>	Mengambil kelas dengan probabilitas tertinggi	Output = kelas dengan skor prediksi paling besar.
<code>cv2.putText(frame, ...)</code>	Menampilkan hasil klasifikasi pada jendela	Label hasil klasifikasi ditulis di atas video.
<code>cv2.imshow('Frame', frame)</code>	Menampilkan citra asli dengan label	Jendela utama prediksi CNN.
<code>cv2.imshow('Night Vision', night_vision)</code>	Menampilkan citra mode <i>Night Vision</i>	Jendela tambahan untuk visibilitas rendah.
<code>cv2.waitKey(1)</code>	Kontrol untuk keluar dengan tombol q	Jika ditekan q, loop berhenti.
<code>cap.release(), cv2.destroyAllWindows()</code>	Menutup kamera dan jendela OpenCV	Membersihkan resource setelah selesai.

6. Data dan Output Hasil Pengamatan

Setelah dilakukan pengujian pada program yang telah dimodifikasi dengan metode **Support Vector Machine (SVM)**, sistem deteksi warna mampu mengenali objek secara real-time melalui kamera. Setiap objek yang terdeteksi diberi bounding box disertai label warna sesuai hasil prediksi, sementara nilai akurasi ditampilkan langsung pada layar. Hasil uji menunjukkan bahwa sistem dapat mengklasifikasikan warna dengan cukup baik, meskipun akurasi masih dipengaruhi oleh kondisi pencahayaan serta kemiripan warna objek dengan latar belakang, sebagaimana ditunjukkan pada tabel hasil pengamatan berikut.

No	Input Citra (Kamera)	Output Night Vision	Hasil Prediksi CNN	Gambar
1	Gambar laut (permukaan air & gelombang)	Citra berwarna semu dengan colormap JET (biru–kuning–merah) yang menonjolkan perbedaan intensitas pada permukaan laut	Class: sea	
2	Gambar pegunungan bersalju (dengan langit di atasnya)	Citra colormap JET menampilkan gradasi biru–kuning–merah pada area gunung dan langit	Class: buildings	
3	Gambar jalan dengan pepohonan di sekitarnya	Citra colormap JET menonjolkan bentuk jalan dan vegetasi sekitar dengan warna semu	Class: forest	

7. Kesimpulan

Dari hasil implementasi dapat disimpulkan bahwa model **Convolutional Neural Network (CNN)** yang digunakan cukup mampu melakukan klasifikasi citra secara real-time. Hal ini terlihat dari keberhasilan dalam mengenali beberapa kelas, seperti *sea* dan *forest*. Meski demikian, masih terdapat kesalahan prediksi, contohnya citra gunung yang terklasifikasi sebagai *buildings*, yang menunjukkan bahwa model masih sulit membedakan kelas dengan karakteristik visual yang mirip. Penerapan mode **Night Vision** menggunakan colormap JET juga membantu meningkatkan keterbacaan citra pada kondisi cahaya rendah, meskipun tidak secara langsung memengaruhi hasil klasifikasi CNN. Secara keseluruhan, sistem ini memiliki potensi untuk digunakan dalam pendeteksian objek lingkungan seperti laut, hutan, jalan, maupun bangunan, meski tingkat akurasi masih perlu ditingkatkan.

8. Saran

Saran yang dapat diberikan antara lain memperluas dataset dengan variasi citra yang lebih beragam agar model mampu membedakan objek dengan kemiripan fitur secara lebih akurat. Dari sisi arsitektur, CNN dapat ditingkatkan atau diganti dengan pendekatan **transfer learning** untuk memperoleh hasil klasifikasi yang lebih optimal. Untuk performa, pemanfaatan GPU atau penerapan model ringan seperti **TensorFlow Lite** dapat mempercepat proses prediksi pada perangkat dengan spesifikasi terbatas. Pengembangan mode **Night Vision** juga disarankan menggunakan metode **low-light enhancement** berbasis deep learning agar citra tampak lebih natural dan tetap jelas meski pada kondisi gelap total. Selain itu, pengujian lanjutan dengan variasi pencahayaan, sudut kamera, serta jenis objek yang berbeda penting dilakukan guna mengevaluasi sejauh mana sistem dapat berfungsi secara efektif pada kondisi nyata.

9. Daftar Pustaka

Liu, F., & Fan, L. (2025). *A review of advancements in low-light image enhancement using deep learning*. arXiv. Artikel ini mengulas berbagai metode deep learning terbaru (sejak 2020) .